

The Latent Cognitive Trilogy

Paulo Leocadio  0000-0002-4992-4541

Zinnia Labs, Beoward, FL, United States

May 17, 2026

Series Preface

The increasing scale and capability of contemporary neural architectures have exposed a persistent gap in their theoretical characterization. While substantial progress has been made in optimizing performance and scaling training regimes, the internal organization of learned representations, particularly the interaction between memory, abstraction, and computation, remains only partially formalized. Existing approaches often treat these components as separable concerns: memory as external retrieval, symbolic reasoning as emergent but unstructured behavior, and architectural routing as an engineering optimization. This fragmentation limits the ability to reason about model behavior in a unified manner.

This work is motivated by the need for a more structured account of internal model dynamics. Rather than introducing a new empirical benchmark or an isolated architectural modification, the present study adopts a conceptual approach to identify and formalize recurring patterns observed across modern neural systems. The goal is not to provide a complete theory, but to establish a minimal set of constructs through which different aspects of model behavior can be described within a shared framework.

The research program introduced here, titled **The Latent Cognition Trilogy**, is organized as a sequence of three investigations, each addressing a distinct but interrelated aspect of internal computation. The first volume examines memory as an intrinsic, differentiable component of the model, proposing a formulation in which retrieval is integrated directly into the latent computational process. The second volume considers the conditions under which compressed representations acquire symbolic stability, with emphasis on the role of curriculum and structural constraints in shaping emergent abstractions. The third volume addresses architectural organization, analyzing how hierarchical routing mechanisms coordinate computation across specialized components to maintain coherence at scale.

Taken together, these studies do not assume a unified theory a priori, but instead develop a layered description of internal model structure. Each

volume isolates a specific class of mechanisms and proposes corresponding formalizations and testable propositions. The progression from memory to symbolic representation to architectural routing is therefore not intended as a claim of completeness, but as a structured pathway for analysis.

This preface serves to situate the individual volumes within that broader program. The sections that follow introduce the foundational constructs and relationships that are shared across the trilogy, providing a consistent point of reference for the detailed developments presented in each volume.

Note on the Research Program

****Note on the Research Program: The Latent Cognition Trilogy****

This article constitutes the first component of a structured theoretical research program aimed at formalizing internal mechanisms of machine cognition. The program, titled **The Latent Cognition Trilogy**, is organized in three stages: (i) the definition of latent memory as a differentiable field within neural architectures, (ii) the emergence of symbolic structure under compression and curriculum constraints, and (iii) the architectural organization of computation through hierarchical routing mechanisms.

Rather than presenting a unified theory, each volume isolates a specific class of mechanisms and develops corresponding formal descriptions and testable propositions. The present work (Volume I) introduces Differentiable Latent Indexing (DLI) as a candidate formalism for internal memory retrieval within transformer-based models.

Subsequent volumes extend this foundation by examining the conditions under which latent representations acquire symbolic stability (Volume II), and how these structures govern large-scale computational organization (Volume III).

Conceptual Foundations

Core Constructs

The trilogy operates on a set of shared constructs that describe internal representations and computational organization in neural architectures.

Latent State. A representation $\mathbf{h}_t \in \mathcal{L}$ encoding the internal state of the model at step t .

Memory Field. A structured collection $\mathcal{M}$ of latent representations accessible through differentiable retrieval.

Symbolic Representation. A compressed encoding $s \in \mathcal{S}$ derived from latent states.

Routing Function. A mapping $\mathcal{G}(\cdot \mid \mathcal{E})$ that governs allocation of computation across specialized modules.

Structural Relationship

The trilogy examines the interaction between these constructs across three stages:

$$\mathcal{M} \implies \mathcal{S} \implies \mathcal{R}$$

VOLUME I: MEMORY FIELDS

*Differentiable Latent Indexing inside Large Language
Models*

Abstract

This article proposes *Differentiable Latent Indexing* (DLI), a theoretical framework that recontextualizes memory retrieval not as an external utility appended to a language model, but as a native, differentiable function inside the computational pathway of Large Language Models (LLMs). By treating latent space as a structured information field, the article derives formal retrieval operators through which memory access can be integrated directly into the model’s forward pass.

The central claim is that the conventional separation between storage and processing introduces architectural inefficiencies that limit the development of autonomous and cognitively organized AI systems. In current retrieval-augmented designs, the model, retriever, embedding store, and generator often remain operationally coupled but mathematically discontinuous. DLI instead proposes a shared latent memory substrate in which retrieval, routing, and representation learning are jointly optimized.

This work is presented as a conceptual research article. It does not claim empirical validation. Instead, it establishes mathematical scaffolding, explicit propositions, falsification conditions, implementation pathways, and expected failure modes for future experimental work. In doing so, it serves as the first component of a broader research program investigating the internal organization of machine cognition.

Keywords: *Differentiable Latent Indexing (DLI); Large Language Models (LLMs); Retrieval-Augmented Generation (RAG); Cognitive Architectures; Neural Retrieval; Differentiable Memory; Transformer Models; Cognitive Complete AI; Hypothesis-Extension Research; Artificial Intelligence Theory.*

Chapter 1

Propositions

Proposition 1: Retrieval Efficiency Gain

If DLI provides a meaningful internal memory mechanism, models augmented with DLI should demonstrate improved performance on tasks requiring long-range dependency resolution compared with baseline transformers of equivalent parameter count.

Falsification condition: If no statistically significant improvement is observed across controlled benchmarks, the hypothesis that internally differentiable retrieval improves effective memory capacity is weakened.

Proposition 2: Latent Clustering Stability

DLI should induce measurable clustering in latent representations corresponding to recurrent semantic structures.

Falsification condition: If latent representations remain uniformly distributed or indistinguishable from baseline attention patterns, the claim of structured internal memory is not supported. To mathematically verify this organization, the spatial distribution of the latent memory field can be monitored via a localized Silhouette Coefficient (S). For a cluster of recurrent semantic structures C , the stability metric is formalized as:

$$S(M^{(\ell)}) = \frac{1}{|C|} \sum_{i \in C} \frac{b(i) - a(i)}{\max(a(i), b(i))}, \quad (1.1)$$

where $a(i)$ represents the mean intra-cluster distance of representation i , and $b(i)$ represents the mean nearest-cluster distance to any other semantic partition. The proposition holds if and only if $S(M^{(\ell)}) > \epsilon$ as training

progress approaches convergence, where ϵ is the baseline uniform distribution threshold.

Proposition 3: Retrieval-Induced Drift

Repeated retrieval operations should introduce measurable drift in latent state trajectories.

Falsification condition: If no measurable deviation from baseline forward-pass dynamics is observed, the proposed retrieval interaction is not functionally distinct.

Introduction

The present work inaugurates a trilogy of conceptual research articles that together outline a research trajectory centered on latent cognition in artificial systems. This first article formalizes *Differentiable Latent Indexing* (DLI) as a native retrieval mechanism inside Transformer-based language models. The article does not report experimental results; rather, it formalizes, elaborates, and critically develops a hypothesis into a coherent architectural framework. In this sense, the work belongs to a theoretical tradition in which formal models precede experimental construction.

The paper extends the Differentiable Latent Indexing inside Large Language Models (DLI-LLM) hypothesis by positioning it as an architectural paradigm for retrieval inside Transformer computation. The second article in this series will examine *Curriculum-Induced Symbol Invention* (CISI) as a mechanism for emergent reasoning primitives. The third article will synthesize both threads into a broader account of how memory and symbols may be integrated in artificial cognition. Together, these papers aim to establish a conceptual lineage for AI systems that treats memory, routing, representation, and control as interdependent components of machine cognition.

This article was developed in response to a hypothesis generated by the Academia.edu AI-assisted scientific modeling platform. The originating hypothesis proposed a learnable, differentiable memory retrieval mechanism embedded directly within Transformer layers. Rather than treating that hypothesis as a fixed endpoint, the present paper treats it as a conceptual seed: a starting point from which to examine the architectural, cognitive, mathematical, and safety implications of internalized retrieval.

The problem space addressed here is the separation between parametric memory, stored within model weights, and non-parametric memory, re-

trieved through external documents, embedding stores, or vector databases. Retrieval-augmented generation has improved factual grounding by combining sequence-to-sequence generation with external dense retrieval [9]. Related approaches, including nearest-neighbor language models and memorizing transformers, demonstrate that language models can benefit from access to stored representations beyond the immediate context window [7, 15]. However, these approaches also preserve a structural separation between the model’s internal computation and the retrieval substrate.

This separation produces three persistent architectural constraints:

1. latency and bandwidth trade-offs between inference and retrieval operations;
2. semantic drift between retrieved information and the evolving internal context of the model; and
3. limited gradient flow between evidence selection, memory access, and downstream reasoning.

DLI proposes a departure from this pattern. It introduces a differentiable memory module embedded within Transformer layers and trained jointly with the model. Memory queries are formulated from hidden states, routed toward a latent key–value memory, and fused back into the model through gated integration. Product quantization provides a possible mechanism for scalable approximate search, while gating regulates when retrieved memory should influence the hidden state [5, 6, 8].

By situating retrieval as an internalized cognitive function, DLI reframes memory access as part of the model’s own computational graph rather than as an external service call. The resulting architecture is not merely an optimization of retrieval-augmented generation. It is a theoretical proposal for unifying attention, memory, and representational control inside Transformer-based systems.

Contributions

This work contributes the following elements:

1. a unified architectural hypothesis that integrates latent memory directly into Transformer computation;
2. a mathematical definition of differentiable retrieval, routing, and gated fusion;

3. a conceptual implementation blueprint for internal retrieval during inference;
4. a comparative analysis connecting DLI to retrieval-augmented generation, kNN language models, differentiable memory systems, recurrent memory transformers, and cognitive architectures; and
5. a foundation for the broader research program developed in the *Latent Cognition Trilogy*.

1.1 Overview of the Paper

Section 2 develops the methodological foundation by decomposing the originating DLI-LLM hypothesis into its constituent architectural claims. Section 3 presents theoretical predictions and falsifiable implications. Section 4 develops the mathematical formulation of Differentiable Latent Indexing. Section 5 provides a conceptual implementation blueprint and comparative literature analysis. Section 6 discusses broader implications for cognitive modeling, learning dynamics, safety, and cognitive-complete Transformer architectures.

1.2 Theoretical Construction and Implications

This paper does not report empirical results. Its contribution is the theoretical development of a Transformer-based architecture through structured hypothesis modeling. The starting point is the DLI-LLM hypothesis, which is decomposed into a set of architectural claims and then expanded into a formal framework suitable for future empirical study.

1.2.1 Hypothesis Decomposition

The original hypothesis can be decomposed into four architectural claims:

1. a differentiable memory module embedded inside Transformer layers;
2. a product-quantized memory structure for scalable approximate retrieval;
3. a shared latent vector space for token-level representations and document-level embeddings; and
4. a learned query-gating mechanism for retrieval triggering and fusion control.

Each claim has precedent in adjacent literature but differs in the proposed integration. Neural Turing Machines and Differentiable Neural Computers introduced differentiable access to external memory [3, 4]. Memory-augmented neural networks demonstrated rapid assimilation of new examples through content-based memory access [12]. Recurrent memory transformers and memorizing transformers later adapted memory mechanisms to Transformer-based language modeling [2, 15]. DLI differs from these systems by treating latent retrieval as an internal and trainable component of the Transformer layer itself.

1.2.2 Architectural Modeling

The hypothesis is extended into a multi-stage retrieval loop inside the forward pass. At a high level, the current hidden state generates a query vector. That query vector retrieves top- k value vectors from a latent key-value memory. The retrieved representation is then fused back into the model through a learned routing gate. This loop creates a differentiable pathway between hidden-state evolution and memory access.

The proposed retrieval loop contains three major operations:

1. cross-layer memory interpolation, in which retrieval results influence subsequent Transformer layers;
2. sparse top- k selection, allowing retrieval to remain computationally bounded; and
3. internal-external evidence arbitration, in which latent memory competes with or complements the local attention pathway.

To visualize the core concept, **Figure 1.1** introduces the proposed DLI-LLM architecture.

1.2.3 Comparative Contextualization

DLI intersects with several existing research lines. Retrieval-augmented generation combines parametric sequence generation with non-parametric document retrieval [9]. kNN-LM interpolates language-model predictions with nearest-neighbor lookup over stored representations [7]. Memorizing Transformers store recent key-value representations for later lookup [15]. Dense passage retrieval systems optimize learned representations for open-domain question answering [11]. Memory-augmented neural networks and

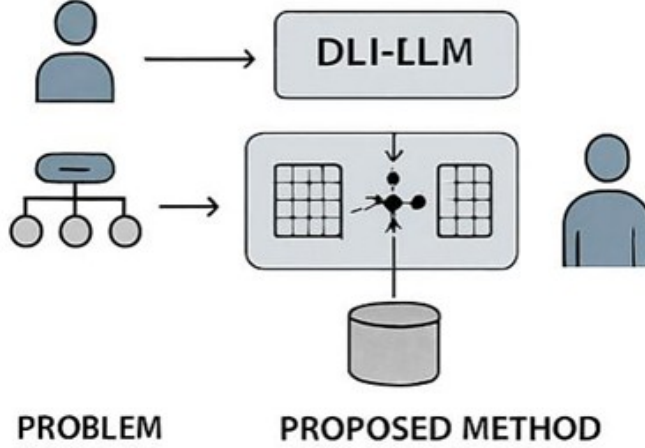


Figure 1.1: Differentiable Latent Indexing architecture. The diagram illustrates how latent key-value memory slots are queried from the current hidden state and fused back into the Transformer attention pathway during the forward pass.

differentiable memory systems demonstrate that memory can be made trainable through differentiable addressing [3, 4, 12].

DLI draws from all of these traditions but modifies the integration point. Rather than positioning retrieval as a pre-generation evidence step or a post-hoc interpolation mechanism, DLI treats retrieval as part of the Transformer layer’s representational dynamics. The implication is architectural: the model does not merely consult memory; it computes through memory.

1.3 Theoretical Predictions

Because no DLI-LLM model is implemented in this article, the following claims should be read as theoretical predictions rather than empirical results. They define expected behaviors that future experimental studies should attempt to confirm, refine, or falsify.

- **Retrieval latency.** By integrating retrieval into the forward pass, DLI-LLM is expected to reduce the external retriever bottleneck characteristic of conventional RAG architectures. Approximate nearest-neighbor methods and GPU-accelerated similarity search provide a

plausible basis for this efficiency claim [5, 6].

- **Grounding stability.** Stable latent memory may reduce hallucination in knowledge-intensive tasks by narrowing the distance between retrieved evidence and internal representation. This prediction is motivated by improvements observed in retrieval-augmented generation and nearest-neighbor language modeling [9, 7].
- **Generalization through memory retention.** Because latent memory slots are differentially integrated into Transformer layers, DLI may reproduce some of the generalization-through-memorization dynamics observed in kNN-LM and memorizing transformers [7, 15].
- **Few-shot adaptation.** Learned gating may improve adaptation under limited supervision by controlling when retrieved memory should alter the hidden state, a behavior related to memory-augmented few-shot learning [12].
- **Cross-layer coherence.** Query propagation across layers may improve long-context coherence by maintaining retrieval continuity over successive transformations, complementing trends in recurrent memory transformers [2].

The predicted failure modes are equally important:

- *Memory saturation:* fixed-size latent memory may become overloaded under high document or representation density [3, 15].
- *Value-vector redundancy:* top- k retrieval in high-dimensional latent spaces may return semantically redundant vectors, reducing effective retrieval precision without pruning, compression, or quantization [5, 6].
- *Context misalignment:* retrieval queries may drift when cross-layer propagation interacts poorly with earlier representations, particularly in recurrent or persistent-memory settings [2].
- *Opacity of internal retrieval:* once retrieval is internalized, the evidentiary path may become harder to inspect than in externally mediated RAG systems.

Figure 1.2 contrasts the conventional retrieval-augmented generation pattern with the internalized retrieval logic proposed by DLI.

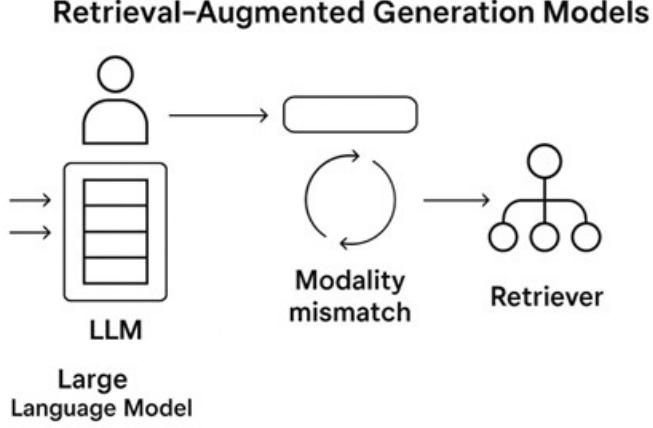


Figure 1.2: Standard retrieval-augmented generation pipeline. In conventional RAG, retrieval remains externally mediated through a document index or vector store before generation. DLI proposes to move part of this retrieval logic into the model’s differentiable computation path.

1.4 Mathematical Framing of Differentiable Latent Indexing

To transition from conceptual architecture to a formally analyzable model, this section expresses DLI-LLM as a minimal extension of standard Transformer notation. Let $h_t^{(\ell)} \in \mathbb{R}^d$ denote the hidden state at token position t and layer ℓ , following the general structure of attention-based Transformer computation [14]. DLI augments this computation with a latent key–value memory and a differentiable routing mechanism.

1.4.1 Latent Key–Value Memory

For each layer ℓ , define a latent memory bank:

$$M^{(\ell)} = \{(k_i^{(\ell)}, v_i^{(\ell)})\}_{i=1}^N, \quad (1.2)$$

where $k_i^{(\ell)} \in \mathbb{R}^{d_k}$ is a memory key and $v_i^{(\ell)} \in \mathbb{R}^{d_v}$ is the corresponding memory value. The memory bank may contain internal token-derived representations, document-derived embeddings, or a hybrid of both. This formulation extends differentiable memory systems by embedding the memory inside the layer-level computation rather than attaching it as a separate

controller [3, 4, 12].

1.4.2 Query Projection

A learned projection maps the current hidden state into a retrieval query:

$$q_t^{(\ell)} = W_q^{(\ell)} h_t^{(\ell)} + b_q^{(\ell)}. \quad (1.3)$$

This query projection is analogous to attention-based addressing but is directed toward persistent latent memory rather than only toward positions within the active context window [1, 14].

1.4.3 Similarity and Sparse Retrieval

The model computes retrieval logits through a similarity function:

$$s_i^{(\ell)} = \frac{q_t^{(\ell)} \cdot k_i^{(\ell)}}{\sqrt{d_k}} \quad (1.4)$$

A sparse top- k selection operator restricts retrieval to the most relevant memory entries:

$$\alpha_i^{(\ell)} = \frac{\exp(s_i^{(\ell)}/\tau) \mathbf{1}[i \in \text{TopK}(s^{(\ell)}, k)]}{\sum_{j \in \text{TopK}(s^{(\ell)}, k)} \exp(s_j^{(\ell)}/\tau)} \quad (1.5)$$

where τ is a temperature parameter and $\alpha_i^{(\ell)}$ is the differentiable retrieval weight assigned to memory slot i . Product quantization may be used to approximate key search at scale [5, 6].

1.4.4 Retrieved Representation

The retrieved latent representation is computed as a weighted sum of memory values:

$$r_t^{(\ell)} = \sum_{i=1}^N \alpha_i^{(\ell)} v_i^{(\ell)}. \quad (1.6)$$

This operation turns memory access into a differentiable aggregation step, preserving gradient flow through the retrieval distribution.

1.4.5 Gated Fusion

A learned gate controls the influence of retrieved memory on the evolving hidden state:

$$g_t^{(\ell)} = \sigma \left(W_g^{(\ell)} [h_t^{(\ell)} \parallel r_t^{(\ell)}] + b_g^{(\ell)} \right), \quad (1.7)$$

where $[h_t^{(\ell)} \parallel r_t^{(\ell)}]$ denotes vector concatenation and σ is the logistic sigmoid. The fused representation is then:

$$\tilde{h}_t^{(\ell)} = \text{LayerNorm} \left(h_t^{(\ell)} + g_t^{(\ell)} \odot r_t^{(\ell)} \right). \quad (1.8)$$

This gating operation connects DLI to conditional computation and mixture-style routing, while preserving a layer-local mechanism for regulating memory influence [8, 2].

1.4.6 Training Objective

A conceptual DLI training objective can be written as:

$$\mathcal{L}_{DLI} = \mathcal{L}_{LM} + \lambda_s \mathcal{L}_{sparse} + \lambda_m \mathcal{L}_{memory} + \lambda_g \mathcal{L}_{gate} \quad (1.9)$$

where $\mathcal{L}_{LM}$ is the standard language modeling loss, $\mathcal{L}_{sparse}$ regularizes retrieval selectivity by penalizing the entropy of the allocation profile α , $\mathcal{L}_{memory}$ regulates memory coherence or redundancy, and $\mathcal{L}_{gate}$ discourages uncontrolled reliance on retrieved values. The exact form of these terms is an empirical design question, but the formulation clarifies where future implementations may intervene.

1.4.7 Gradient Flow and Optimization Continuity

Because the sparse selection operator $\text{TopK}(\cdot)$ introduces a non-differentiable boundary, gradient continuity through the retrieval pathway is maintained via the soft-max weighting allocation across the chosen partition. The partial derivative of the fused representation $\tilde{h}_t^{(\ell)}$ with respect to the memory keys $k_i^{(\ell)}$ propagates directly through the similarity scaling:

$$\frac{\partial \tilde{h}_t^{(\ell)}}{\partial k_i^{(\ell)}} = g_t^{(\ell)} \odot v_i^{(\ell)} \left(\frac{\partial \alpha_i^{(\ell)}}{\partial s_i^{(\ell)}} \right) \left(\frac{q_t^{(\ell)}}{\sqrt{d_k}} \right). \quad (1.10)$$

This formalizes a continuous, uninterrupted backward pass from downstream language modeling errors directly into the latent memory coordinates,

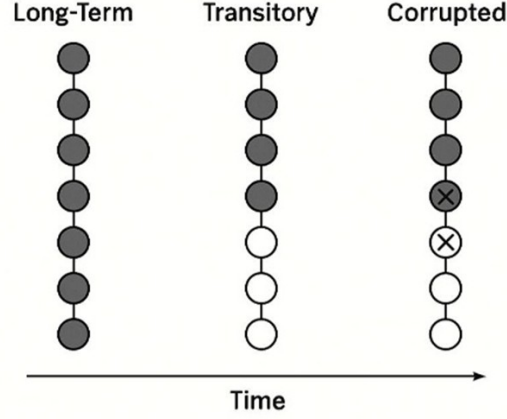


Figure 3. Lifespan topologies: stable, transitory, and polluted.

Figure 1.3: Gradient pathways in Differentiable Latent Indexing. Query projection, similarity computation, sparse retrieval weighting, routed memory integration, and gated fusion remain within the differentiable computation graph, allowing latent memory to co-evolve with Transformer parameters.

allowing the index structure to co-evolve with parametric transformations without a separate optimization loop.

Figure 1.3 illustrates how gradients propagate through the attention and retrieval pathways during training.

1.5 Algorithmic Specification and Implementation Blueprint

To operationalize the retrieval mechanism described above, Algorithm 1.5 specifies a single DLI retrieval step within one Transformer layer.

Algorithm 1: Differentiable Latent Indexing Loop

Input: hidden state $h_t^{(\ell)}$, latent memory $M^{(\ell)} = \{(k_i^{(\ell)}, v_i^{(\ell)})\}_{i=1}^N$, parameters $W_q^{(\ell)}, W_g^{(\ell)}$, top- k value, temperature τ .

Output: fused representation $\tilde{h}_t^{(\ell)}$.

1. Compute retrieval query: $q_t^{(\ell)} \leftarrow W_q^{(\ell)} h_t^{(\ell)} + b_q^{(\ell)}$.
2. Compute similarity scores: $s_i^{(\ell)} \leftarrow q_t^{(\ell)} \cdot k_i^{(\ell)} / \sqrt{d_k}$.
3. Select top- k memory slots according to $s_i^{(\ell)}$.
4. Compute retrieval weights: $\alpha^{(\ell)} \leftarrow \text{softmax}(s^{(\ell)} / \tau)$ over selected slots.
5. Retrieve latent value: $r_t^{(\ell)} \leftarrow \sum_i \alpha_i^{(\ell)} v_i^{(\ell)}$.
6. Compute routing gate: $g_t^{(\ell)} \leftarrow \sigma(W_g^{(\ell)} [h_t^{(\ell)} \parallel r_t^{(\ell)}] + b_g^{(\ell)})$.
7. Fuse retrieved memory with hidden state: $\tilde{h}_t^{(\ell)} \leftarrow \text{LayerNorm}(h_t^{(\ell)} + g_t^{(\ell)} \odot r_t^{(\ell)})$.
8. Return $\tilde{h}_t^{(\ell)}$.

Although this paper is theoretical, an implementable DLI prototype would require four design stages.

1.5.1 Memory Construction

Document embeddings, token representations, or intermediate hidden states would be transformed into latent key-value banks. Product quantization could then be applied to compress memory and enable approximate retrieval at scale [5, 6].

1.5.2 Quantized Latent Indexing Mechanics

To prevent the TopK operations from scaling linearly $\mathcal{O}(N)$ with the size of the latent memory repository, the key space is structured using Product Quantization (PQ). The high-dimensional key space $\mathbb{R}^{d_k}$ is decomposed into M orthogonal subspaces:

$$\mathbb{R}^{d_k} = \mathbb{R}^{d_s} \times \mathbb{R}^{d_s} \times \dots \times \mathbb{R}^{d_s}, \quad (1.11)$$

where $d_s = d_k/M$. Each subspace is quantized into K^* centroids using a learned codebook. During the forward pass, the query projection $q_t^{(\ell)}$ is similarly sliced into M sub-vectors. Similarity estimation $s_i^{(\ell)}$ is then executed via asymmetric distance computation (ADC) using pre-calculated look-up tables. This reduces the computational overhead of the internalized retrieval phase to a collection of parallel cache lookups, maintaining a highly bounded inference budget.

1.5.3 Layer Integration

A DLI block would be inserted after self-attention or between attention and feed-forward sublayers. The routing gate would determine whether retrieved values should influence the hidden state. This placement allows DLI to function as a memory-aware extension of Transformer computation rather than as a separate retrieval pipeline.

1.5.4 Training and Fine-Tuning

Training would jointly optimize language modeling, retrieval selectivity, memory coherence, and gating behavior. In a pre-training setting, memory slots could evolve with the model. In a fine-tuning setting, selected memory banks could be frozen, swapped, or domain-adapted.

1.5.5 Inference Behavior

During inference, DLI could operate through frozen or partially updateable memory. Sparse top- k retrieval would limit computational overhead. Memory swapping could support domain-specific deployment, while gating thresholds could regulate when latent memory is allowed to influence generation.

1.5.6 Evaluation Plan

A future empirical study should compare DLI-augmented models against baseline Transformers, RAG models, kNN-LM variants, and memorizing transformers. Evaluation should include latency overhead, retrieval precision, hallucination rates, long-context coherence, memory saturation, and ablation studies that disable routing, memory access, or quantization independently.

1.6 Comparative Literature Analysis

Table 1.1 summarizes the position of DLI relative to adjacent architectures.

Architecture	Relation to DLI	Limitation Relative to DLI
Retrieval-Augmented Generation	Combines parametric generation with non-parametric retrieval [9].	Retrieval is typically external to the Transformer computation graph.
k NN-LM	Interpolates language-model predictions with nearest-neighbor lookup [7].	Retrieval is post-hoc and not trained as an internal layer function.
Memorizing Transformers	Store and retrieve recent key-value representations [15].	Memory improves context use but remains distinct from document-level latent unification.
Neural Turing Machines / DNC	Demonstrate differentiable memory addressing [3, 4].	Designed as general memory controllers rather than layer-native LLM retrieval modules.
Memory-Augmented Neural Networks	Support rapid assimilation of new examples [12].	Primarily oriented toward few-shot learning rather than Transformer-internal retrieval.
Recurrent Memory Transformer	Uses memory tokens for long-sequence processing [2].	Memory persistence is present, but explicit latent key-value retrieval control is limited.
Dense Passage Retrieval / RocketQA	Optimizes learned representations for retrieval [11].	Retrieval occurs outside the model’s internal forward computation.
Product-Quantized Vector Search	Provides scalable approximate search [5, 6].	Supplies retrieval infrastructure but not a cognitive or Transformer-layer architecture.

Table 1.1: Comparative analysis of memory-augmented and retrieval-integrated architectures.

1.7 Discussion

The DLI-LLM architecture reframes how memory is conceptualized in Transformer-based systems. By integrating retrieval into the forward pass, the model collapses the traditional separation between reasoning and recall. This aligns with cognitive architecture research that treats memory, control, learning, and representation as interdependent properties of intelligent systems [13].

1.7.1 Cognitive Implications

Embedding differentiable latent memory directly within Transformer layers positions retrieval as a native cognitive operation rather than an external API call. This shift parallels earlier insights from memory-augmented neural networks and differentiable memory systems [3, 4, 12], but extends them by proposing a unified latent space where token-derived representations and document-derived vectors can coexist. The resulting architecture moves toward cognitive completeness by enabling context-dependent recall, internal evidence arbitration, and persistent cross-layer memory flow.

1.7.2 Learning Dynamics and Adaptation

DLI-LLM suggests new possibilities for lifelong and continual learning. If memory slots can be updated incrementally, the model may integrate new documents without full retraining, a capability related to recurrent memory systems and neural semantic encoders [2, 10]. However, this advantage introduces challenges. Latent memory may overfit internal distributions, memory slots may saturate, and retrieval may become redundant without pruning, quantization, or regularization [5, 6].

1.7.3 Bias, Safety, and Privacy Considerations

The integration of retrieval into model computation raises safety concerns. Frozen memory layers may preserve sensitive information, creating privacy risks. Internal retrieval may reinforce biased representations. Latent memories, once implicitly learned, may be harder to inspect than externally retrieved documents. These risks parallel issues in retrieval-augmented systems but are amplified by the opacity of internalized memory [9].

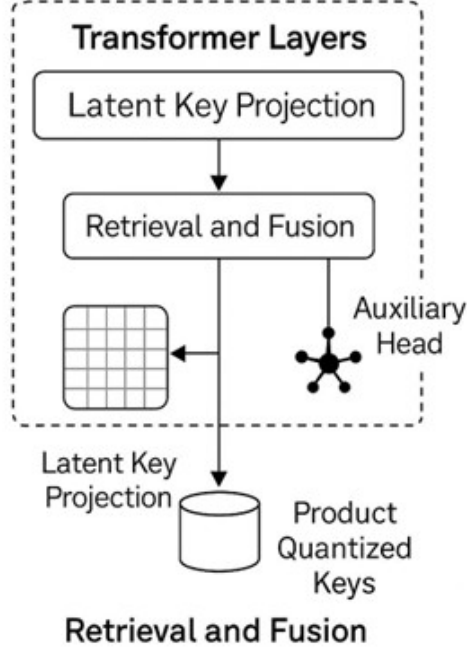


Figure 1.4: Training flow and gating control in DLI-LLM. Product-quantized key–value lookup, attention fusion, routing gates, and gradient-based memory adaptation interact across Transformer blocks.

1.7.4 Architectural Control and Gating Mechanisms

A central feature of DLI-LLM is the routing gate. The gate mediates the integration of retrieved values into the hidden state, determines how retrieval influences inference, and regulates cross-layer propagation. Conceptually, this places gating at the core of cognitive control: the architecture must learn not only what to retrieve, but also when retrieval should matter. **Figure 1.4** illustrates this control path.

1.7.5 Toward Cognitive-Complete Transformer Architectures

DLI-LLM argues for reconsidering memory-augmented Transformers not as storage-bound enhancements, but as components of a broader approach to machine cognition. In this view, retrieval is not merely a way to extend context length; it is a candidate mechanism for internal evidence selection, representational persistence, and adaptive control. This aligns with the

broader desideratum that cognitive architectures should integrate memory, learning, representation, and control into a coherent system [13].

1.8 Conclusion

Differentiable Latent Indexing proposes that retrieval can be moved from the periphery of language-model architecture into the differentiable interior of Transformer computation. The framework introduced here formalizes this idea through latent key–value memory, query projection, sparse retrieval, gated fusion, and a conceptual training objective. It also defines falsifiable propositions, expected benefits, and plausible failure modes.

The paper does not claim that DLI-LLM has already been implemented or empirically validated. Its contribution is theoretical: it clarifies what such an architecture would require, how it differs from existing retrieval and memory-augmented models, and what future experiments would need to test. As the first volume in the *Latent Cognition Trilogy*, this article establishes memory as the initial boundary condition for a broader investigation into artificial cognition.

VOLUME II: SYMBOL FIELDS

*Curriculum-Induced Symbol Invention for Emergent
Reasoning Primitives*

Abstract

This article introduces *Curriculum-Induced Symbol Invention* (CISI), a theoretical framework exploring the organic synthesis of discrete reasoning primitives within the intermediate pathways of Large Language Models (LLMs). Standard Transformer architectures suffer from representational passivity, absorbing tokens sequentially without compiling recurring computational sub-routines into invariant abstractions. We propose an architectural modification wherein hidden state trajectories are intercepted and compressed through an auxiliary, vector-quantized latent codebook layer during pre-training. Driven by an engineered curriculum of increasing task complexity and a strict informational bottleneck, the model is forced to coin, stabilize, and reuse its own internal tokens—effectively developing a private, domain-specific language (DSL) optimized for computational economy and symbolic execution.

Building upon the native, internalized memory substrate established in *Volume I: Memory Fields*, this work shifts the focus from continuous storage to discrete abstraction. CISI treats symbol coining not as a post-hoc linguistic mapping, but as a structural survival mechanism for networks operating under intense transmission constraints. This work functions as an independent, self-contained specification and foundational design pattern, delineating explicit training stages, codebook optimization dynamics, semiographic translation probes, and formal empirical evaluation protocols for emergent machine semiotics.

Keywords: *Curriculum-Induced Symbol Invention (CISI); Latent Semiography; Vector Quantization; Information Bottleneck; Domain-Specific Languages; Machine Cognition; Structural Representation Learning; Automated Abstraction.*

Chapter 2

Propositions and Falsification Canon

The validity of Curriculum-Induced Symbol Invention as a standalone cognitive engine rests upon explicit, predictable behavioral deviations from standard Transformer baselines. The following formal propositions establish the empirical boundaries of the architecture and dictate clear falsification parameters.

Proposition 1: Symbolic Abstraction Efficiency

As structural curriculum complexity scales, an LLM computational graph constrained by a discrete intermediate vector-quantization bottleneck will demonstrate significantly greater cross-domain generalization and sample efficiency in out-of-distribution (OOD) reasoning tasks compared to standard continuous-state architectures of equivalent parameter density.

Falsification Condition: If a continuous-state baseline matches or exceeds the CISI architecture on downstream multi-step reasoning evaluations when controlled for training token exposure and active parameter count, the claim that discrete intermediate symbol coining resolves structural compositionality is falsified.

Proposition 2: Codebook Reuse and Structural Sparsity

The optimization trajectory of a bounded latent codebook operating under variable curriculum demands will favor the stabilization of a minimal, highly reusable set of primitive codewords over a strategy of continuous, uniform codebook resource expansion.

Falsification Condition: If tracking codebook utilization profiles reveals a linear growth path proportional to task variation, or if the codebook selection entropy approaches maximum uniform distribution without distinct topological clustering, the hypothesis that the informational bottleneck forces symbolic compression is ungrounded.

Proposition 3: Degradation under Bottleneck Deactivation

The deactivation or continuous relaxation of the intermediate quantization layer following curriculum exposure will result in a catastrophic degradation of out-of-distribution reasoning capabilities, while local in-distribution language modeling performance remains unaffected.

Falsification Condition: If an operational CISI model maintains its advanced multi-step reasoning capabilities when the vector quantization layer is bypassed or replaced with an unconstrained linear projection, the symbolic utility of the discrete codebook is disproven.

Chapter 3

The Informational Bottleneck and Vector Quantization

Let $h_t^{(\ell)} \in \mathbb{R}^d$ define the continuous hidden representation of a token at sequence position t and intermediate layer ℓ within the standard Transformer forward pass. The CISI architecture systematically intercepts this trajectory, passing the continuous vector field through a discrete quantization bottleneck before it propagates to subsequent layers.

3.0.1 Vector Quantization Mechanics

We define an internal, learned discrete codebook dictionary $\mathcal{C} = \{e_1, e_2, \dots, e_K\}$, where each individual codeword vector $e_k \in \mathbb{R}^d$. The continuous latent state $h_t^{(\ell)}$ is mapped directly to its nearest symbolic primitive within the dictionary via a minimum Euclidean distance projection operator:

$$z_t^{(\ell)} = \text{vquant}\left(h_t^{(\ell)}\right) = e_{\hat{k}}, \quad \text{where } \hat{k} = \arg \min_k \|h_t^{(\ell)} - e_k\|_2. \quad (3.1)$$

The selected codebook index $\hat{k}$ serves as the discrete, organic latent symbol invented by the architecture to represent that particular continuous computational sub-manifold.

3.0.2 The Compression Objective

To enforce intense compression pressure and protect the network against representational collapse or passive passing-through of uncompressed tokens, an auxiliary decoder head $\mathcal{D}$ is introduced. This decoder is tasked with

reconstructing the complete, high-dimensional continuous hidden state $h_t^{(\ell)}$ using exclusively the quantized discrete vector assignment $z_t^{(\ell)}$:

$$\mathcal{L}_{\text{comp}} = \|\mathcal{D}\left(z_t^{(\ell)}\right) - \text{sg}[h_t^{(\ell)}]\|_2^2 + \beta \|\text{sg}[h_t^{(\ell)}] - z_t^{(\ell)}\|_2^2, \quad (3.2)$$

where $\text{sg}[\cdot]$ represents the classic stop-gradient operator required to prevent unstable backward oscillations during joint optimization, and β is a commitment hyperparameter regulating the alignment rate between continuous trajectories and codebook coordinates.

Chapter 4

Staged Curriculum Design

The autonomous invention of logical primitives cannot successfully execute within unstructured, high-entropy text environments. It requires an evolutionary gradient. The CISI training framework utilizes a multi-tiered curriculum designed to systematically trigger symbol creation by varying operational complexity.

Curriculum Stage	Operational Complexity	Expected Symbolic Emergence
Stage 1: Primitives	Single-digit arithmetic, basic relational logic statements, atomic parsing scripts.	Coining of baseline primitives (e.g., latent equivalents to functional terms like "ADD", "COMPARE").
Stage 2: Composition	Compound functional routines, multi-hop dependencies, variable tracking loops.	Combination of Stage 1 codewords into complex, integrated computational blocks.
Stage 3: Integration	Knowledge-intensive text, open-domain reasoning benchmarks (GSM8K, CommonsenseQA).	Synthesis of a robust, highly optimized internal Domain-Specific Language (DSL).

Table 4.1: The CISI Staged Training Curriculum Profile.

During Stage 1, a minimal partition of 10 to 20 active codewords suffices for error minimization. As the model transitions to Stage 2 and Stage 3, the codebook is permitted to expand under strict sparsity regularizers, forcing previous symbols to act as structural building blocks for higher-order reasoning routines.

Chapter 5

Semiographic Projection and Interpretation

Once symbols stabilize within the intermediate codebook substrate, they must be rendered inspectable to ensure cognitive accountability and alignment. CISI avoids back-correcting the network via human prose; instead, it reads the machine’s internal language directly.

5.0.1 Linear Probing and Linguistic Translation

We introduce a periodic probing schedule using linear classification models applied directly to index activation frequencies over standardized validation partitions:

$$P(T_m \mid e_{\hat{k}}) = \text{softmax} \left(W_p e_{\hat{k}} + b_p \right), \quad (5.1)$$

where T_m represents a human-interpretable linguistic concept or operational token. This translation layer functions as a forensic tool, decoding raw latent cluster identifiers into legible intermediate reasoning trajectories, thereby exposing the inner paths of the machine’s deductive logic.

Chapter 6

Empirical Evaluation Protocol

To validate the theoretical assertions of Curriculum-Induced Symbol Invention, any experimental implementation must undergo a rigid, multi-layered forensic protocol consisting of four primary validation operations.

6.0.1 Zero-Shot Out-of-Distribution Evaluation

The primary benchmark of structural symbol creation is the model’s ability to execute zero-shot transfers to entirely novel reasoning datasets. Following exposure to the staged curriculum, the CISI architecture must be evaluated on logic and relational datasets whose underlying semantic distributions share zero overlap with the training inputs. Performance metrics must be strictly compared against baseline Transformer models of identical parameter counts and token exposure.

6.0.2 Ablation of the Compression Interface

To prove that the discrete codebook is the active engine behind reasoning stability, developers must execute a structural ablation pass. By systematically disabling the compression layer post-training—or dynamically substituting the vector quantization operator with an unconstrained, continuous linear layer—the evaluator must track the exact delta in generalization performance. A corresponding drop in out-of-distribution accuracy confirms the operational necessity of the symbols.

6.0.3 Topological Clustering and Code-Reuse Metrics

The spatial distribution of the codebook must be mathematically evaluated using the localized Silhouette Coefficient (S) formalized in the canon of Volume I. Evaluators must verify that the continuous hidden states collapse into highly concentrated, discrete geometric regions surrounding active codebook centroids. Furthermore, the codebook utilization profile must manifest a Power Law distribution, confirming high code-reuse across disparate tasks.

6.0.4 Human Semiotic Mapping and Consistency Studies

Finally, the cognitive validity of the invented internal symbols must be tested through a blind human annotation study. Human evaluators are presented with isolated activation maps of specific codewords alongside their corresponding linguistic contexts decoded via the linear probing translation matrix. Annotators are tasked with mapping these active indices to precise English descriptions or operational rules. The degree of semantic agreement across annotators is quantified using Fleiss' Kappa (κ), verifying whether the machine's invented symbols correspond to stable, invariant concepts.

VOLUME III: ROUTING FIELDS

*Hierarchical Topic-Conditioned Routing for
Mixture-of-Expert Language Models*

Abstract

This article introduces *Hierarchical Topic-Conditioned Routing* (HTCR), an architectural framework designed to resolve token-level dispersion and expert cache locality bottlenecks within sparse Mixture-of-Experts (MoE) language models. Standard MoE architectures process routing protocols independently for each token, ignoring document-level semantic patterns and structural context. This token-isolated approach causes extreme expert jitter, limits sequence-level cache reuse, and complicates distributed hardware scaling. We propose a two-tier routing mechanism: a macroscopic document-router dynamically assigns local context segments (N tokens) to a specialized, restricted pool of m experts; subsequently, a microscopic token-router dispatches individual tokens exclusively within that pre-allocated shortlist.

Building upon the memory substrates of *Volume I* and the symbolic abstractions of *Volume II*, HTCR closes the trilogy by defining the dynamic macroscale routing infrastructure of the network. This work serves as an independent reference design pattern, detailing the mathematical mechanics of two-stage dispatch, structural load-balancing metrics, training stability regularizers, and empirical validation protocols for distributed cognitive architectures.

Keywords: *Hierarchical Topic-Conditioned Routing (HTCR); Mixture-of-Experts (MoE); Cache Locality; Distributed Inference; Expert Jitter; Token Dispatch; Structural Regularization.*

Chapter 7

Propositions and Falsification Canon

The architectural deployment of Hierarchical Topic-Conditioned Routing rests upon verifiable, predictable behavioral variances against flat token-routed MoE baselines (such as standard Mixtral or GShard configurations).

Proposition 1: Topological Cache Locality and Throughput

By constraining individual token execution trajectories to a segment-bounded expert shortlist, the HTCR framework will demonstrate a significant reduction in inter-device KV-cache communication volume and yield higher token processing throughput under long-context sequence execution.

Falsification Condition: If an HTCR model manifests equivalent or higher inter-node communication latency and lower generation throughput compared to an unconstrained token-MoE baseline when evaluated on continuous context lengths exceeding 32k tokens, the utility of segment-level clustering is falsified.

Proposition 2: Optimization Stability and Balances

The combination of macro-level entropy regularizers and token-level gates will stabilize expert assignment trajectories, preventing severe expert starvation and balancing computational loads across distributed hardware partitions without sacrificing modeling capacity.

Falsification Condition: If tracking routing distributions during large-scale pre-training reveals persistent structural load imbalances, or if token-drop ratios match flat gating methods under maximum hardware throughput limits, the optimization balancing claims are ungrounded.

Chapter 8

The Dual-Stage Hierarchical Routing Architecture

Let a long-context sequence be partitioned into discrete segments S_j containing N sequential tokens, where the input token matrix is defined as $X \in \mathbb{R}^{N \times d}$.

8.0.1 Stage 1: Macroscopic Segment Routing

Before token-level evaluation begins, the local hidden representations from the previous layer are aggregated into a unified segment-level summary vector $s_j \in \mathbb{R}^d$ via strategic mean-pooling across the spatial dimension:

$$s_j = \frac{1}{N} \sum_{t=1}^N h_t^{(\ell-1)}. \quad (8.1)$$

This summary vector s_j is processed by a document-router $\mathcal{R}_{\text{doc}}$, parameterized by a two-layer Multi-Layer Perceptron (MLP), to compute an allocation profile over the entire pool of E total available experts:

$$G_{\text{doc}}(s_j) = \text{softmax}(W_2 \cdot \text{GeLU}(W_1 s_j + b_1) + b_2). \quad (8.2)$$

The top- m indices matching the highest activation values in $G_{\text{doc}}(s_j)$ are extracted to define the restricted active expert shortlist $\mathcal{M}_j = \{i_1, i_2, \dots, i_m\}$ for the duration of segment S_j .

8.0.2 Stage 2: Microscopic Token Dispatch

For each individual token representation $h_t^{(\ell)}$ residing within segment S_j , the local token-router $\mathcal{R}_{\text{tok}}$ evaluates gating scores *exclusively* against the experts preserved in the shortlist $\mathcal{M}_j$:

$$G_{\text{tok}}\left(h_t^{(\ell)}\right)=\text{softmax}\left(\left\{w_k^{\top} h_t^{(\ell)} \mid k \in \mathcal{M}_j\right\}\right) . \quad (8.3)$$

Individual tokens are then dispatched to the top- k experts selected strictly from this local dictionary. Gradients flow end-to-end through both routing layers simultaneously via continuous backpropagation paths.

Chapter 9

Structural Regularization and Starvation Prevention

To prevent the macroscopic document-router from collapsing into a biased state where a subset of experts absorbs all workloads (starvation), we enforce an auxiliary balanced regularization penalty $\mathcal{L}_{\text{bal}}$ during pre-training:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda_1 \mathcal{L}_{\text{entropy}} + \lambda_2 \mathcal{D}_{\text{KL}}(\bar{G}_{\text{doc}} \parallel \mathcal{U}), \quad (9.1)$$

where $\bar{G}_{\text{doc}}$ is the average macro-routing distribution across the training batch, $\mathcal{U}$ represents a uniform distribution over the expert pool E , and hyperparameters λ_1, λ_2 govern the structural smoothing constraints. The shortlist ceiling parameter m is dynamically annealed throughout training, starting wide to maximize early discovery and tightening progressively to optimize final execution speed.

Chapter 10

Empirical Evaluation Protocol

To validate the scalability and efficiency claims of the HTCR architecture, tracking scripts must execute four foundational forensic operations against equivalent baseline setups.

10.0.1 Distributed Throughput and Communication Profiles

The system must be evaluated on a distributed compute cluster across multiple GPU nodes. Scaling metrics must explicitly monitor the cross-device data volume transferred via standard **All-to-All** communication primitives. HTCR must demonstrate a significant reduction in communication overhead and near-linear text generation throughput scaling as context window lengths increase.

10.0.2 Annealing Path and Capacity Ablations

Evaluators must isolate the performance changes triggered by the schedule of the shortlist constraint parameter m . Systematic comparative training runs must trace the delta between a static un-annealed baseline and a dynamic scheduling path to map out the explicit convergence characteristics of the dual routers.

10.0.3 Expert Activation Entropy and Balancing Verification

The long-term distribution profiles of the expert array must be audited using standard structural allocation tracking. The load distribution profile

must approximate a uniform target, proving that the balancing penalty effectively prevents hardware starvation without degrading perplexity metrics on standard benchmarks like MMLU and GSM8K.

10.0.4 Cache Locality and Memory Footprint Mapping

Finally, hardware trackers must register real-time L2/L3 cache hit-rates and high-bandwidth memory usage metrics during processing blocks. The HTCR architecture must achieve significant cache retention benefits, showing that document-level conditioning prevents standard expert cache eviction loops.

Bibliography

- [1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *International Conference on Learning Representations (ICLR)*, 2015.
- [2] Aydar Bulatov, Yuri Kuratov, and Mikhail Burtsev. Recurrent memory transformer. *Advances in Neural Information Processing Systems*, 35:11079–11091, 2022.
- [3] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural Turing Machines. *arXiv preprint arXiv:1410.5401*, 2014.
- [4] Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, 2016.
- [5] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.
- [6] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.
- [7] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Zettlemoyer Luke, and Kevin Clark. Generalization through memorization: Nearest neighbor language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.
- [8] Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng

- Chen. GShard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.
- [9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Lewis, Sebastian Riedel, and Tim Rocktäschel. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
 - [10] Tsendsuren Munkhdalai and Hong Yu. Neural semantic encoders. *Association for Computational Linguistics (ACL)*, pages 397–407, 2017.
 - [11] Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Wayne Pan, Hua Wu, and Haifeng Wang. RocketQA: An optimized training approach to dense passage retrieval for open-domain question answering. *Conference of the National Chapter of the Association for Computational Linguistics (NAACL)*, 2021.
 - [12] Adam Santoro, Sergey Bartunov, Matthew Botvinick, Daan Wierstra, and Timothy Lillicrap. Meta-learning with memory-augmented neural networks. *International Conference on Machine Learning (ICML)*, pages 1842–1850, 2016.
 - [13] Ron Sun. Desiderata for cognitive architectures. *Philosophical Psychology*, 17(3):327–353, 2004.
 - [14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30:5998–6008, 2017.
 - [15] Yuhuai Wu, Markus Rahnama, Angela Fan, Sainbayar Sukhbaatar, Jason Weston, and Armand Joulin. Memorizing transformers. *International Conference on Learning Representations (ICLR)*, 2022.